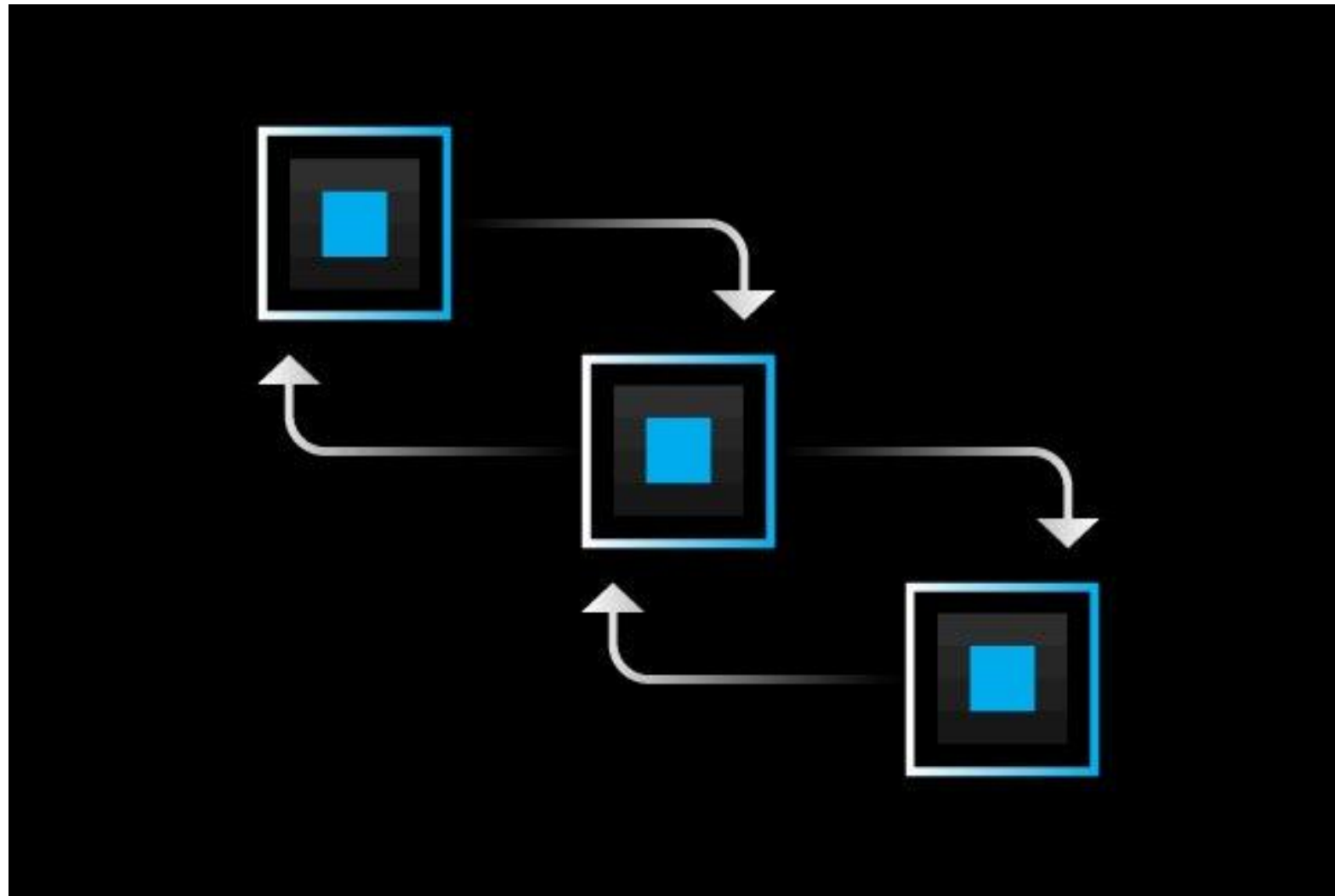




PELO FUTURO DO TRABALHO

Gerenciamento de Versionamento Eficiente

Versionamento



Versionamento

No cenário de desenvolvimento de software atual, é praticamente obrigatório o uso de ferramentas de versionamento, seja para código, configurações ou para documentação em geral (LOELIGER; MCCULLOUGH, 2012).

O uso de sistemas e técnicas de versionamento e revisão de versões se tornaram essenciais para o desenvolvimento de software atualmente. Tais sistemas são encontrados em três diferentes paradigmas: sistemas de versionamento local, sistemas de controle de versão distribuídos - **DVCS** - e sistemas de controle de versão centralizados – **CVCS** (ZOLKIFLI AMIR NGAH*, 2018).

Version Control Software - VCS

O controle de versão é essencial para equipes de desenvolvimento em todos os setores. Ele não apenas permite que eles gerenciem alterações no código e nos arquivos ao longo do tempo, mas também permite que trabalhem no mesmo projeto simultaneamente. Um bom sistema de controle de versão permite melhor colaboração e desenvolvimento mais rápido, além de fornecer um histórico completo de seus ativos digitais.

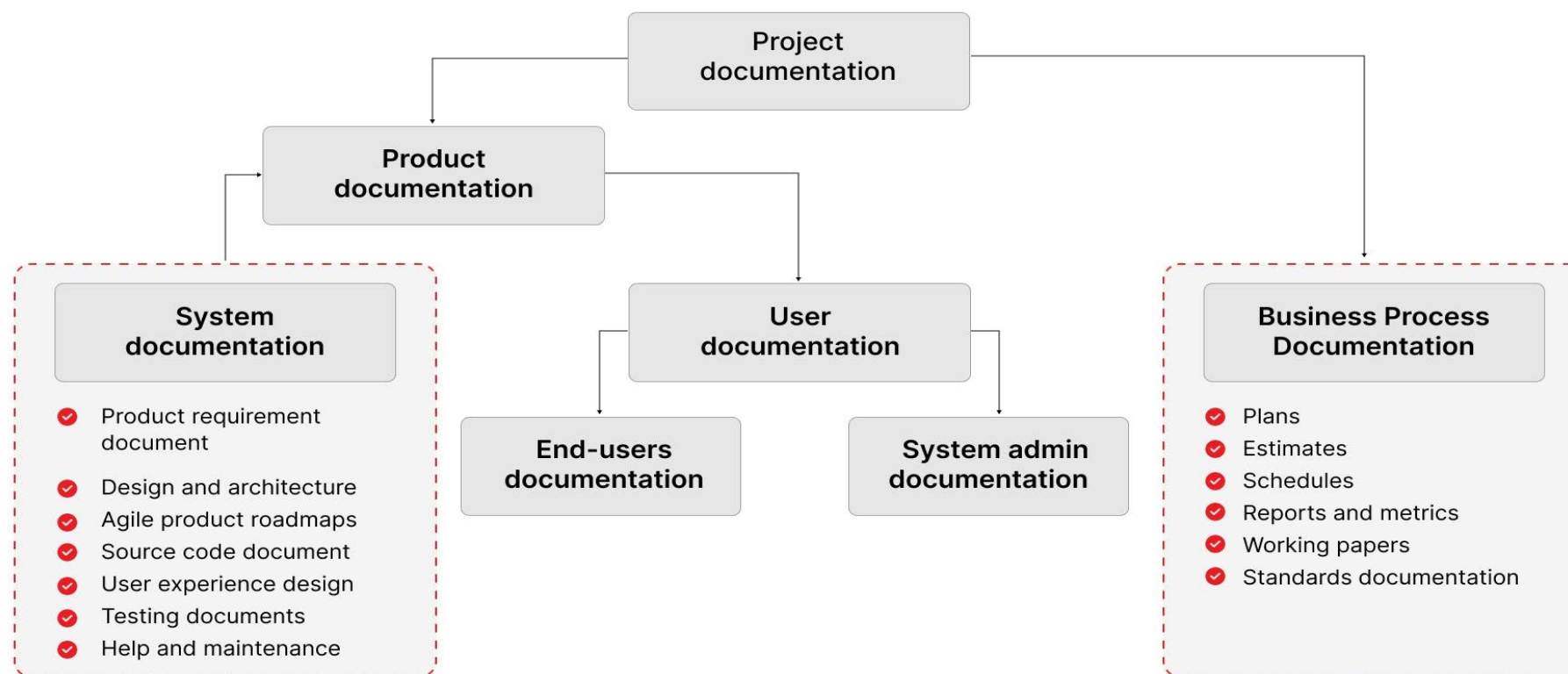
Vantagens

Usando o controle de versão correto:

- Melhora a visibilidade.
- Minimiza conflitos de arquivo e esforço desperdiçado.
- Ajuda as equipes a colaborar em todo o mundo.
- Acelera a entrega do produto.

Tipos de documentação

SOFTWARE DOCUMENTATION TYPES



Importância

O controle de versão é importante para acompanhar as alterações no código, arquivos e outros ativos digitais. Você deve usar o software de controle de versão para todos os ativos e projetos de desenvolvimento nos quais vários membros da equipe colaborarão.

O controle de versão precisa fazer mais do que apenas gerenciar e rastrear arquivos. Deve ajudá-lo a desenvolver e enviar produtos mais rapidamente. Isso é especialmente importante para fluxos de trabalho de DevOps em que a entrega das versões mais recentes pode ser testada ou criada durante o envio da alteração. Bons sistemas de controle de versão sinalizarão possíveis conflitos antes que eles entrem na linha principal do projeto. Isso significa que o desenvolvedor pode corrigir o problema e não promover o problema posteriormente.

Tipos: Centralizado X Distribuído

Os dois tipos mais comuns de sistemas de controle de versão são centralizados e distribuídos. O controle de versão centralizado armazena todas as alterações em um local central. Com o controle de versão distribuído, cada membro da equipe faz alterações em uma cópia local do repositório em seu computador.

CVCS – Centralized version Control System

Em um sistema de controle de versão centralizado (CVCS), um único servidor armazena todas as versões e alterações em todos os arquivos. Com esse tipo de controle de versão, os usuários podem fazer check-out de um arquivo para trabalhar nele e fazer alterações antes de devolvê-lo ao servidor central. Ou eles podem criar uma ramificação separada do projeto, trabalhar nela sozinha e mesclá-la novamente na ramificação principal. Todas as alterações — todos os commits e merges — são sincronizadas com o servidor central.

Um dos principais benefícios de um VCS centralizado é que cada usuário tem acesso rápido à versão mais recente de um arquivo. Isso reduz muito a possibilidade de sobrescrever o trabalho um do outro, duplicar esforços ou desperdiçar esforços fazendo alterações em arquivos que não podem ser mesclados (como arquivos binários).

DVCS – Distributed version Control System

Com um sistema de controle de versão distribuído (DVCS), cada usuário baixa uma cópia do repositório para poder fazer alterações nos arquivos localmente (em sua própria máquina). Existe um repositório principal, mas em vez de armazenar todos os arquivos e todas as suas alterações, esse repositório principal apenas salva as diferenças entre os arquivos (exceto no caso de arquivos binários).

Uma vantagem desse tipo de controle de versão é que pode ser mais rápido para os usuários confirmarem as alterações do que alguns sistemas de controle de versão centralizado. Também permite que os usuários trabalhem offline.

Ferramentas

- Helix Core
- Git
- SVN
- ClearCase
- Mercurial
- Plastic SCM/Unity Version Control
- TFS

Benefícios de controle de versão

Benefícios

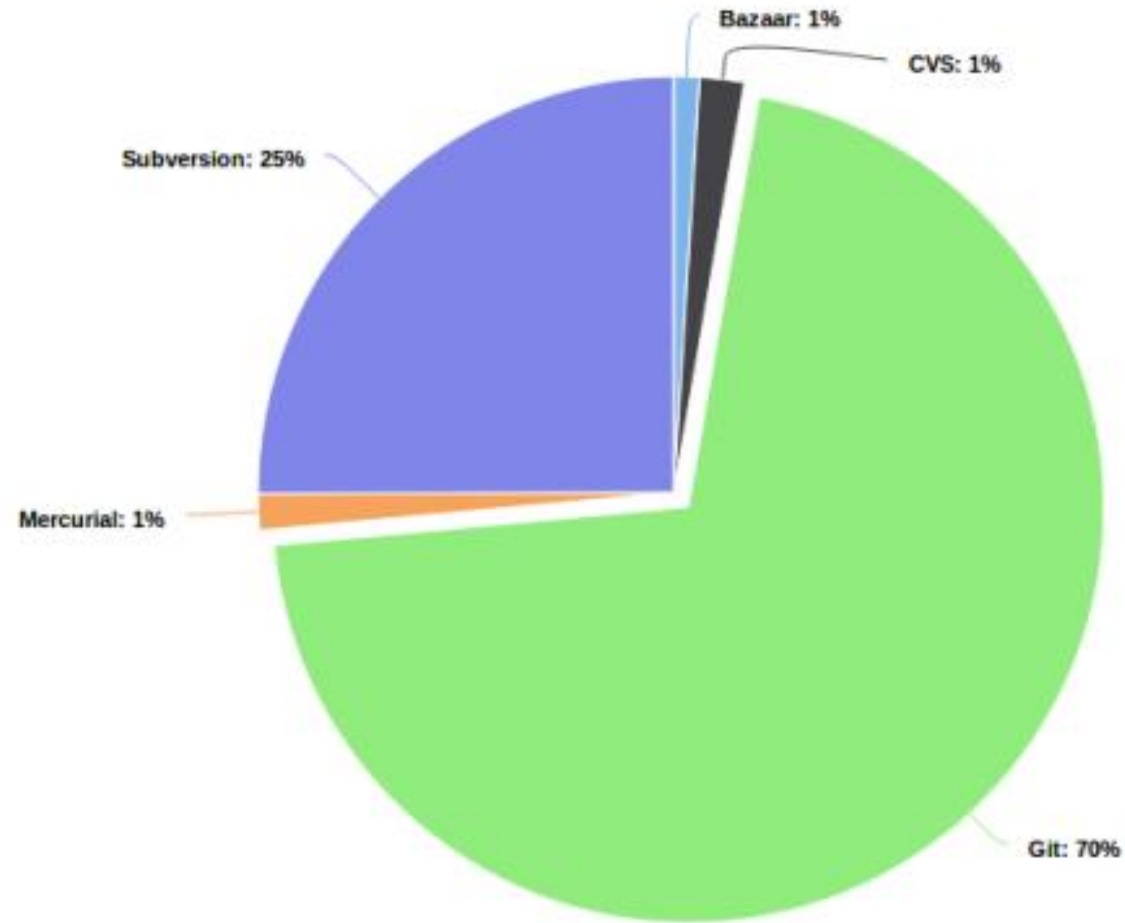
- Centralização: O controle de versão oferece à sua equipe uma única fonte de verdade.
- Governança, risco e Compliance: O controle de versão mantém um histórico completo do arquivo e melhora a visibilidade.
- Desenvolvimento paralelo: O controle de versão permite o desenvolvimento simultâneo.
- Controle: O controle de versão permite uma melhor colaboração em equipe.
- Automação: O controle de versão oferece suporte à automação no desenvolvimento

Benefícios

- Colaboração e trabalho em equipe
- Acompanhamento de Alterações e Revisões
- Backup e recuperação de desastres
- Integridade do código e controle de qualidade

Documentação

Comparação de número de repositórios com o Git e outras tecnologias



Objetivo

Com tamanha diversidade de padrões, convenções e formatos, é comum que exista bastante inconsistência dentro dos repositórios e projetos de software. Este fato pode ocorrer por vários fatores, sendo o principal deles a adaptação à um contexto diferente de desenvolvimento.

Tal adaptação é custosa de tempo e a adaptabilidade necessária de um desenvolvedor à um contexto de trabalho é um requisito do mercado cada vez mais cobrado. Essa adaptação se torna mais custosa na mesma proporção da quantidade de regras e políticas adotadas na equipe de desenvolvimento.



O Git é uma ferramenta bastante poderosa e de fácil uso em sua forma básica, porém seu uso pode se tornar bastante complexo ao se usar funcionalidades mais avançadas. O objetivo desta seção é introduzir elementos e conceitos presentes no restante do trabalho, para garantir um melhor entendimento dos conceitos.

Seja individualmente ou de forma colaborativa, o uso da ferramenta Git gira em torno de três funções fundamentais: commits, branches e interações com outros repositórios. Cada uma dessas permite, respectivamente, o versionamento, o trabalho em paralelo e o controle de versionamento distribuído.

Funcionamento VCS

Organização

- 1- Um repositório é criado, onde os arquivos e todas as suas versões serão armazenados ao longo do tempo.
- 2- À medida que as alterações são feitas nos arquivos armazenados no repositório, elas são rastreadas automaticamente para que você possa ver o que foi alterado, quando e por quem.
- 3- Se um membro da equipe quiser trabalhar em um arquivo, ele pode verificar o arquivo, fazer alterações e, em seguida, confirmar essas alterações/verificá-las no repositório assim que estiverem concluídas. Fazer check-out do arquivo dessa maneira garante que outras pessoas da equipe saibam que ele está sendo trabalhado, o que os impede de substituir as alterações dos outros. Após o check-in das alterações, a nova versão do arquivo mais recente fica disponível para o restante da equipe.

Organização

4- Se um membro da equipe quiser fazer alterações em um projeto sem afetar outros usuários, por exemplo, se quiser experimentar um novo recurso, ele poderá criar uma nova ramificação. Esta é essencialmente uma cópia do projeto. Eles podem trabalhar isoladamente em suas alterações em sua própria ramificação e, em seguida, mesclá-la novamente na ramificação principal.

Terminologias

Terminologias

- **Repositório:** Um repositório é um local centralizado onde todo o código-fonte e os arquivos relacionados de um projeto são armazenados.
- **Commit:** Um commit é uma operação em que um desenvolvedor registra as alterações feitas em um ou mais arquivos dentro do repositório.
- **Branch:** Um branch é uma ramificação independente do código em um repositório. Quando um novo branch é criado, ele herda todo o histórico de commits do branch principal (geralmente chamado de "master" ou "main").

Terminologias

- **Merge:** O merge é o processo de combinar as alterações feitas em um branch com outro. Geralmente, isso significa incorporar as alterações de um branch secundário de volta ao branch principal.
- **Conflito (Conflict):** Um conflito ocorre quando duas ou mais alterações incompatíveis são feitas no mesmo trecho de código ou arquivo por desenvolvedores diferentes, em diferentes branches. Ao tentar mesclar (merge) os branches, o sistema de controle de versão não consegue determinar automaticamente qual versão deve prevalecer.

Fundamentos da web



Pesquisa

- Grupos de 4 alunos
- Desenvolver uma apresentação sobre o funcionamento da ferramenta

Atividade

- Criar uma conta no GitHub

Atividade 2

- https://www.alura.com.br/artigos/iniciando-repositorio-git?gclid=EAlaIQobChMI0Kyw_-SngAMVS-ZcCh1ueACaEAAYASAAEgJDEfD_BwE

Atividade 3

O que é versionamento de software e qual é a sua importância no desenvolvimento de projetos?

Quais são as principais vantagens de utilizar sistemas de controle de versão, como o Git, em projetos de desenvolvimento de software?

Quais são os principais sistemas de controle de versão disponíveis atualmente?

Como funciona o processo de criação e gerenciamento de um repositório em um sistema de controle de versão?

O que são commits e como eles são úteis para controlar as mudanças em um projeto?

Como os branches são utilizados no versionamento de software? Quais são os benefícios de trabalhar com branches em projetos de desenvolvimento?

O que é uma mesclagem (merge) no contexto do versionamento de código? Como ela é realizada?

Quais são os principais desafios que podem surgir ao trabalhar com vários desenvolvedores em um mesmo projeto e como o versionamento de software pode ajudar a superá-los?

Quais são as melhores práticas para escrever mensagens de commit significativas e úteis?

Como resolver conflitos de mesclagem (merge) que ocorrem quando dois ou mais desenvolvedores modificam o mesmo trecho de código?



PELO FUTURO DO TRABALHO

sesisc.org.br     **sc.senai.br**

0800 048 1212

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC